

多级 CPU 张量并行分解方案技术调研报告

核心结论

在多 CPU 架构下实现张量并行的两级分解，核心思路是将原本单一维度的张量并行扩展为跨 NUMA QPI（高速域）和跨 CXL（低速域）的两个正交并行维度。通过在隐藏维度（hidden dimension）和注意力头维度（attention heads）上同时进行分层切分，使得高频通信被限制在 QPI 域内完成，而 CXL 域仅承担低频聚合通信。这种方案本质上是将 2D 张量并行（SUMMA 算法思想）映射到异构网络拓扑上，通过行-列分层切分实现通信量的最优分配。

1. 张量并行技术基础与演进

1.1 一维张量并行（Megatron-LM 1D TP）

一维张量并行是分布式大模型训练中最基础且应用最广泛的模型并行技术，最早由 Megatron-LM 系统提出并实现。其核心思想是将 Transformer 层中的线性层权重矩阵沿着单一维度进行切分，使得多个计算设备可以协同完成原本需要在单一设备上执行的大型矩阵乘法运算。具体而言，对于一个标准的 Transformer MLP 模块，包含两个核心线性变换：第一个线性层将隐藏状态从维度 h 投影到更高的维度 $4h$ （即 $Y = X \cdot W_1$ ，其中 $W_1 \in \mathbb{R}^{h \times 4h}$ ），经过激活函数处理后，第二个线性层将结果从 $4h$ 投影回 h 维度（即 $Z = \text{GeLU}(Y) \cdot W_2$ ，其中 $W_2 \in \mathbb{R}^{4h \times h}$ ）。在一维张量并行中，第一个权重矩阵 W_1 沿着列维度被切分为 $W_1^{(1)}, W_1^{(2)}, \dots, W_1^{(t)}$ ，每个设备 i 持有其中的一块 $W_1^{(i)} \in \mathbb{R}^{h \times (4h/t)}$ 。当输入 X 被广播到所有设备后，每个设备独立计算 $Y^{(i)} = X \cdot W_1^{(i)}$ ，这一计算不需要设备间通信，因为每个设备处理的是独立的输出通道子集。随后，各设备对 $Y^{(i)}$ 应用激活函数得到 $\text{GeLU}(Y^{(i)})$ 。第二个权重矩阵 W_2 则沿着行维度被切分为 $W_2^{(1)}, W_2^{(2)}, \dots, W_2^{(t)}$ ，其中设备 i 持有 $W_2^{(i)} \in \mathbb{R}^{(4h/t) \times h}$ ，恰好与 $\text{GeLU}(Y^{(i)})$ 的维度匹配，因此每个设备可以计算局部输出 $Z^{(i)} = \text{GeLU}(Y^{(i)}) \cdot W_2^{(i)}$ 。由于 W_2 的行切分，每个设备的 $Z^{(i)}$ 只是最终结果的一部分，需要通过一个 All-Reduce 操作（具体为求和）将所有设备的部分结果聚合，得到最终输出 $Z = \sum_{i=1}^t Z^{(i)}$ 。这种“列切分 + 行切分”的组合巧妙地使得每个 Transformer 层只需要两次 All-Reduce 操作（一次在 MLP 模块，一次在 Attention 模块），大大减少了通信开销。然而，一维张量并行存在显著的局限性：它假设所有参与并行的设备之间具有均匀的通信带宽，这在实际的异构网络拓扑中往往不成立。当张量并行组跨越不同节点或不同互连层级的设备时，通信性能会受到最慢链路的严重制约，导致整体效率急剧下降。此外，随着张量并行度的增加，All-Reduce 的通信量并不会减少（因为激活值在所有设备上完全复制），而有效带宽反而会因更多参与方而降低，使得通信开销成为瓶颈。

对于自注意力（Self-Attention）模块，一维张量并行采用类似的策略。多头注意力中的 Query、Key、Value 投影矩阵沿着列维度（即注意力头维度）进行切分，每个设备负责一部分注意力头的计算。由于不同注意力头之间的计算是完全独立的，这种方式可以自然地并行化。注意力计算完成后，输出投影矩阵 W_O 沿着行维度切分，最后通过 All-Reduce 聚合各部分结果。这种设计使得注意力模块的通信模式与 MLP 模块保持一致，整个 Transformer 层的前向传播和反向传播都遵循统一的通信原语。值得注意的是，一维张量并行在反向传播阶段也需要进行 All-Reduce 操作来聚合梯度，因此每个训练步骤中每个 Transformer 层实际上需要进行四次 All-Reduce 操作（前向两次，反向两次）。尽管 Megatron-LM 通过精心的流水线设计将部分通信与计算重叠，但在大规模部署中，通信开销仍然占据显著比例，尤其是在跨节点场景下。研究表明，在某些配置中，通信可以占据总延迟的 65% 以上，这凸显了优化张量并行通信模式的重要性。

1.2 二维张量并行（SUMMA 算法）

二维张量并行代表了对一维方法的重要扩展，它将计算设备组织成一个二维网格（mesh），并沿着两个不同的维度对张量进行切分。这种方法基于经典的 SUMMA（Scalable Universal Matrix Multiplication Algorithm）算法，该算法在传统的 HPC 领域已被广泛研究和优化。在二维张量并行中，假设有 N 个设备 arranged 成一个 $\sqrt{N} \times \sqrt{N}$ 的网格，输入张量 $X \in \mathbb{R}^{P \times Q}$ 被切分成形状为 $\mathbb{R}^{P/\sqrt{N} \times Q/\sqrt{N}}$ 的小块，分布在各个设备上。与一维张量并行不同，二维方法同时切分输入激活值、权重矩阵和输出激活值，因此在理想情况下可以完全消除内存冗余——每个设备只持有总参数的 $1/N$ ，而不是一维方法中的 $1/\sqrt{N}$ 。二维张量并行的核心通信模式包括 All-Gather、Reduce-Scatter 和 All-Reduce，但这些操作被限制在二维网格的行或列上执行，而不是涉及所有设备。具体来说，在执行矩阵乘法时，SUMMA 算法通过迭代的方法

式在网格的行和列之间广播数据块，使得每个设备在每次迭代中都能获得所需的输入数据进行局部计算，最后通过 reduce 操作汇总结果。

SUMMA 算法的通信成本显著低于一维方法。对于在 P 个处理器上执行的矩阵乘法，一维张量并行的通信复杂度为 $O(P)$ ，而二维 SUMMA 算法的通信复杂度降低到 $O(\sqrt{P})$ 。这一改进来自于更均衡的数据分布和更局部的通信模式。然而，二维张量并行也对设备拓扑提出了更严格的要求：它需要设备能够组织成规则的二维网格，并且行内和列内的通信都需要较高的带宽支持。在实际部署中，这意味着二维张量并行通常适用于具有全连接或接近全连接拓扑的节点内部（如通过 NVSwitch 连接的 8 个 GPU），而在跨节点场景下则面临拓扑不匹配的挑战。此外，SUMMA 算法的实现复杂度也高于一维方法，需要仔细地管理数据布局和通信调度。Colossal-AI 框架实现了二维张量并行，并展示了在某些配置下相比一维方法可以获得更好的内存效率和通信性能。对于 CPU NUMA 架构而言，二维张量并行的思想尤为重要，因为 NUMA 节点本身就可以被自然地组织成二维拓扑——同一 NUMA 节点内的核心构成行或列，而不同 NUMA 节点之间则形成另一个维度。

1.3 2.5D 与 3D 张量并行

2.5D 和 3D 张量并行是对二维方法的进一步推广，它们引入了额外的并行维度来进一步降低通信复杂度和提高可扩展性。2.5D 张量并行由 Solomonik 等人提出，灵感来源于 2.5D 矩阵乘法算法。在这种方法中， P 个处理器被组织为 $P = Q^2 \cdot D$ 的结构，其中 Q 是二维网格的边长， D 是“深度”维度。当 $D = 1$ 时，2.5D 张量并行退化为标准的 2D SUMMA 算法；当 $D > 1$ 时，系统在深度维度上复制了 D 份数据副本，允许更多的并行计算和更少的通信。这种“以空间换时间”的策略通过增加内存占用来减少通信量，为系统在内存容量和通信开销之间提供了灵活的权衡。具体而言，2.5D 算法通过将矩阵分块并在深度维度上分配不同的乘法任务，使得每个处理器只需要与同一深度层内的其他处理器通信，而深度层之间的通信则通过 reduce 操作完成。这种分层通信模式使得 2.5D 张量并行在某些拓扑结构下（如具有层次化互连的系统）能够取得比 2D 更好的性能。

3D 张量并行则更进一步，基于 Johnson 的 3D 矩阵乘法算法，将张量在一个三维处理器立方体上进行切分。对于 N 个设备，张量被切分为形状 $\mathbb{R}^{P/\sqrt[3]{N^2} \times Q/\sqrt[3]{N}}$ 的块，实现了在 N 个处理器上 $O(N^{1/3})$ 的通信复杂度，这在理论上是渐进最优的。3D 张量并行同时对输入张量的第一个和最后一个维度进行切分（其中第一个维度被切分两次），通过精心设计的数据布局和通信调度，使得计算和内存使用能够均匀地分布在所有设备上。Colossal-AI 框架集成了 3D 张量并行，并证明在处理超大规模模型时，3D 方法相比 1D 和 2D 方法具有更好的可扩展性。然而，3D 张量并行对设备数量有严格的限制（需要是完美立方数），且实现复杂度最高，需要精密的负载均衡和通信优化。对于 CPU 多级互连架构而言，2.5D 和 3D 张量并行提供了重要的理论指导：它们展示了如何通过引入额外的抽象维度来匹配层次化的硬件拓扑，从而为设计跨 QPI 和跨 CXL 的两级张量并行方案奠定了理论基础。

1.4 序列并行与上下文并行

序列并行（Sequence Parallelism）和上下文并行（Context Parallelism）是专门针对长序列场景优化的并行技术，它们与张量并行密切相关但解决的是不同维度的问题。在标准的张量并行中，输入序列在所有设备上完全复制，这在处理长序列时会导致激活值内存急剧增长。序列并行通过在序列维度上切分激活值来解决这一问题，使得 LayerNorm、Dropout 等操作的激活值内存占用可以随着并行度线性降低。具体而言，序列并行将输入张量沿着序列长度维度切分，每个设备只处理一部分 token。由于 LayerNorm 和 Dropout 等操作是在序列维度上独立应用的，这种切分不会影响计算的正确性。然而，注意力计算需要全局的序列信息，因此序列并行通常需要与 Ring Attention 等技术结合，通过环形通信模式来计算注意力分数。序列并行可以被视为张量并行在序列维度上的自然扩展，两者结合可以在隐藏维度和序列维度上同时实现并行，从而处理极长的上下文。

上下文并行则是序列并行的一种更高级的形式，专门针对注意力计算的二次复杂度问题。在标准的多头注意力中，计算复杂度与序列长度的平方成正比，这使得长序列成为训练的主要瓶颈。上下文并行通过将注意力计算分布在多个设备上，每个设备只计算一部分注意力分数，然后通过 All-Gather 或 All-to-All 操作交换中间结果。DeepSpeed-Ulysses 是一种代表性的上下文并行方法，它沿着序列维度切分输入，使用 All-to-All 集体通信确保每个 GPU 获得完整的序列但只有一部分注意力头的信息，从而实现注意力计算的并行化。对于 CPU 多级架构而言，序列并行和上下文并行提供了额外的设计维度：短序列的通信可以在高速 QPI 域内完成，而长序列的跨 NUMA 通信则可以通过 CXL 进行，利用 CXL 的内存语义来

优化大规模激活值的传输。

张量并行策略	切分维度	通信复杂度	内存冗余	设备要求	适用场景
1D TP	隐藏维度（列/行）	$O(P)$	高	任意数量	节点内 NVLink/高速互连
2D TP	隐藏维度 + 批维度	$O(\sqrt{P})$	低	需为完全平方数	规则二维网格拓扑
2.5D TP	隐藏维度 + 批维度 + 深度	$O(\sqrt{P/D})$	可调	$P = Q^2 \cdot D$	内存充足、通信受限
3D TP	三维立方体切分	$O(P^{1/3})$	最低	需为完全立方数	超大规模、全连接拓扑
序列并行	序列长度维度	额外 All-to-All	低	任意数量	长序列场景

2. CPU NUMA 架构与 CXL 互连技术

2.1 NUMA 架构与 QPI/UPI 互连特性

非统一内存访问（NUMA）架构是现代多路服务器的基础设计范式。在 NUMA 系统中，每个 CPU 插槽（socket）拥有独立的本地内存控制器和直连的 DRAM 通道，形成所谓的 NUMA 节点。处理器访问本地内存时享有最低的延迟和最高的带宽，而访问其他插槽的远程内存则需要通过插槽间互连链路进行传输。Intel 平台采用的 QuickPath Interconnect（QPI）和 Ultra Path Interconnect（UPI）以及 AMD 平台的 Infinity Fabric 构成了这些互连链路的核心。以典型的双路服务器为例，本地内存访问延迟约为 80-100 纳秒，而跨插槽的远程内存访问延迟则增加到 140-200 纳秒，即所谓的 NUMA 效应，延迟差距约为 1.5 到 2 倍。在带宽方面，单个插槽通过多个 DDR 内存通道可以提供约 50-100 GB/s 的本地内存带宽，而 QPI/UPI 链路的理论带宽通常在 25.6 GB/s 到 41.6 GB/s 之间（取决于具体版本和链路数量），实际可用远程带宽约为本地带宽的 40-70%。

在多路服务器中，QPI/UPI 拓扑对张量并行的设计具有决定性影响。对于直接相连的两个插槽（单跳），数据 traverses 一条 QPI/UPI 链路，延迟约 150-200 纳秒，可用带宽约 10-25 GB/s。而对于需要通过中间插槽转发的多跳场景（如四路服务器中的对角插槽），延迟可能增加到 250-400 纳秒，有效带宽进一步下降。实测数据显示，在典型的四路 Nehalem-EX 系统中，本地内存读取的最大聚合带宽为 24.7 GB/s（延迟约 150 纳秒），单跳远程读取带宽为 10.9 GB/s（延迟约 185 纳秒），而多跳远程读取带宽仅为 5.3 GB/s（延迟约 235 纳秒）。这种显著的带宽和延迟梯度意味着张量并行策略必须充分感知 NUMA 拓扑，将高频通信限制在同一 NUMA 节点或单跳域内。对于张量并行而言，All-Reduce 操作的性能直接受限於最慢的参与链路，因此在跨越 NUMA 边界时，通信效率会急剧下降。

2.2 CXL 技术架构与性能特征

Compute Express Link（CXL）是一种基于 PCIe 物理层的新兴开放互连标准，旨在实现处理器、加速器和内存扩展设备之间的高速、低延迟、缓存一致的通信。CXL 协议栈包含三个子协议：CXL.io 用于设备发现和配置（功能等价于 PCIe），CXL.cache 允许设备缓存主机内存，以及最关键的 CXL.mem，它使主机能够通过标准的 load/store 指令访问设备连接的内存。CXL.mem 子协议是实现内存池化和扩展的核心，它允许将外部内存设备（Type-3 设备）的内存映射到主机的物理地址空间中，操作系统将其视为一个特殊的 NUMA 节点（无 CPU 核心）。从性能角度看，CXL 内存访问延迟取决于多种因素，包括 CXL 控制器实现（ASIC vs FPGA）、PCIe 版本、是否经过交换机等。根据多项研究，本地 CXL 内存（直接连接到同一 CPU 插槽）的访问延迟约为 170-400 纳秒，比本地 DRAM（80-100 纳秒）高约 2-4 倍，但与传统远程 NUMA 访问（140-200 纳秒）处于同一数量级。XConn CXL 2.0 交换机的 64 字节 I/O 延迟约为 658 纳秒，而 CXL 3.0 支持多级交换结构，可以将延迟控制在数百纳秒到 1 微秒级别。

CXL 的带宽特性同样关键。CXL 1.1/2.0 基于 PCIe 5.0（32 GT/s），单个 x16 链路可提供约 64 GB/s 的原始带宽（考虑编码开销后约 50-60 GB/s 的有效带宽）。CXL 3.0 升级到 PCIe 6.0（64 GT/s），x16 链路带宽翻倍至约 128 GB/s。即将推出的 CXL 4.0 将基于 PCIe 7.0（128 GT/s），进一步提升带宽。然而，实际 CXL 设备可用的带宽通常低于理论峰值，因为受到 CXL 控制器、内存介质和协议开销的限制。实测数据显示，CXL 设备的有效带宽约为本地

DRAM 的 40-80%，saturate 于重负载的多线程场景。对于张量并行而言，CXL 提供了一个重要的新能力：它允许将远程内存以接近 NUMA 远程访问的延迟和合理的带宽纳入统一的地址空间，这为设计跨节点的两级张量并行提供了硬件基础。通过 CXL，可以将模型参数存储在远程内存池中，而计算则在本地 CPU 上进行，从而实现计算和内存的解耦。

2.3 QPI/UPI 与 CXL 的性能对比

在设计两级张量并行方案时，深入理解 QPI/UPI 和 CXL 的性能差异至关重要。从延迟角度看，QPI/UPI 直连的远程 NUMA 访问约为 140-200 纳秒，而本地 CXL 访问约为 170-300 纳秒，两者处于相近的水平。然而，当 CXL 访问需要跨越 NUMA 边界（即 CPU 通过 UPI 访问另一插槽的 CXL 设备）时，延迟会显著增加到 350-600 纳秒，因为数据需要 traverses UPI 链路和 CXL 链路两个环节。这种”NUMA+CXL”的组合效应在某些工作负载下可能导致意外的性能下降，甚至不如直接访问远程 NUMA 内存。从带宽角度看，QPI/UPI 的原始带宽约为 25-50 GB/s 每链路，而 CXL x16 (PCIe 5.0) 可提供约 64 GB/s，两者在同一数量级。但实际可用带宽受多种因素影响：QPI/UPI 带宽会被所有跨插槽流量共享（包括缓存一致性流量、I/O 流量等），而 CXL 带宽则受限於 PCIe 链路的实际利用率。

互连类型	典型延迟	有效带宽	缓存一致性	主要用途
本地 DRAM	80-100 ns	50-100 GB/s	硬件一致	主内存
QPI/UPI 直连	140-200 ns	20-50 GB/s	硬件一致	插槽间互联
本地 CXL	170-300 ns	20-60 GB/s	CXL.mem 一致	内存扩展
UPI+CXL	350-600 ns	10-25 GB/s	混合	跨 NUMA 内存访问
CXL+Switch	400-800 ns	32-512 GB/s 聚合	CXL 协议	多主机内存池化

这种层次化的延迟和带宽特征为两级张量并行提供了天然的映射关系：**第一级张量并行（TP-1）应在 QPI/UPI 域内完成**，利用较低的延迟和较高的带宽进行频繁的 All-Reduce 同步；**第二级张量并行（TP-2）则通过 CXL 连接**，利用 CXL 的内存池化能力扩大可访问的内存容量，同时通过减少跨 CXL 的同步频率来摊销较高的通信延迟。这种分层设计的关键在于识别哪些通信操作可以限制在 QPI 域内，哪些必须跨越 CXL 边界，并据此调整张量的切分策略和同步粒度。

2.4 CPU 集群上的张量并行实践

在 CPU 平台上进行大规模模型训练和推理已经获得了显著的关注，尤其是在成本效益和硬件可获取性方面。Intel Xeon 处理器通过 Advanced Matrix Extensions (AMX) 指令集提供了对 bfloat16 和 INT8 数据类型的硬件加速，使得 CPU 上的 LLM 推理性能大幅提升。Intel 的 IPEX (Intel Extension for PyTorch) 和 oneDNN 库针对 CPU 架构进行了深度优化，支持 AVX-512 VNNI 和 AMX 指令。在实际部署中，CPU 上的张量并行通常采用 DeepSpeed 的集成方案，结合张量并行和流水线并行来适配多 NUMA 架构。例如，Intel 的验证参考方案在双路 Xeon 服务器上使用自动张量并行 (Auto Tensor Parallelism)，通过 Sub-NUMA Clustering (SNC) 设置将张量并行组映射到不同的 NUMA 节点，以最大化内存本地性和缓存利用率。

vLLM 等推理框架也针对 CPU 进行了优化，实现了 NUMA 感知的张量并行。这些框架在启动时自动检测系统的 NUMA 拓扑（通过 lscpu 等工具），然后根据 NUMA 节点的数量和每个节点的核心数选择最优的并行策略。例如，当每个插槽的 NUMA 节点数为 2 或 4 时（对称配置），系统倾向于使用张量并行，因为可以均匀地分配核心；而当 NUMA 节点数为 3 或 6 时（非对称配置），则可能选择流水线并行。这种自动化的拓扑感知策略旨在最小化跨 NUMA 流量，最大化内存本地性和缓存利用率。对于 CXL 内存池化的支持，Linux 内核已经将 CXL 内存集成到 NUMA 框架中，用户可以通过 numactl 和 libnuma 工具手动控制内存分配策略。然而，研究表明，简单的交错（interleave）策略在长上下文 LLM 微调等场景下表现次优，需要开发更智能的 CXL 内存分配算法来优化性能。

3. 两级张量并行分解方案设计

3.1 核心设计思想

在多 CPU 架构下设计两级张量并行的核心挑战在于如何将单一的并行维度分解为两个正交的子维度，使得每个子维度的通信模式与其对应的互连层级相匹配。本方案提出基于 2D 张量并行 (SUMMA 算法) 的分层映射策略：将原本在单一维度上切分权重的 1D 张量并行，扩展为在隐藏维度 (hidden dimension) 和注意力头维度 (attention heads) 两个正交方向上的 2D 切分。具体而言，系统将计算设备组织成一个逻辑二维网格，其中网格的行对应于通过 QPI/UPI 高速互连的 NUMA 节点组 (第一级并行, TP-1)，网格的列对应于通过 CXL 互连的多个 NUMA 节点 (第二级并行, TP-2)。这种设计的核心洞察在于：高频同步操作 (如每个 Transformer 层内的 All-Reduce) 被限制在 QPI 域内完成，而低频的聚合操作 (如跨节点的梯度同步或激活值交换) 则通过 CXL 进行。

这种分层设计的优势在于最大化利用了不同互连层级的特性。QPI/UPI 提供低延迟 (约 150-200 纳秒)、高带宽 (20-50 GB/s) 的通信能力，非常适合张量并行中频繁的小规模同步操作。而 CXL 虽然延迟较高 (350-600 纳秒)，但提供了内存池化和更大的聚合带宽 (通过多通道和交换机)，适合作为参数存储和低频大规模数据传输的通道。通过将 2D 网格的行映射到 QPI 域，列映射到 CXL 域，系统可以确保在行方向上执行 All-Reduce 时，数据 traverses 最快的互连路径；而在列方向上，通信量被精心设计为最小化 (主要通过激活值的局部累加而非全量传输)。这种映射关系本质上是对 SUMMA 算法在异构网络拓扑上的适配，通过调整切分维度与硬件拓扑的对齐方式，实现通信代价的最小化。

3.2 两级 MLP 模块并行方案

对于 Transformer 中的 MLP 模块，两级张量并行采用分层切分策略。假设系统包含 N 个 NUMA 节点，每个节点内部通过 QPI/UPI 全连接或部分连接，节点之间通过 CXL 交换机互联。我们将所有计算资源组织为一个二维网格，网格尺寸为 $R \times C$ ，其中 R 是行数 (对应 QPI 域内的 NUMA 组数)， C 是列数 (对应 CXL 域内的 NUMA 节点数)，且 $R \times C = N$ 。对于 MLP 的第一个线性层 $W_1 \in \mathbb{R}^{h \times 4h}$ ，权重矩阵在两个维度上同时切分：在行方向 (QPI 域) 上， W_1 沿着输入隐藏维度 h 被切分为 R 份；在列方向 (CXL 域) 上， W_1 沿着输出扩展维度 $4h$ 被切分为 C 份。这样，每个 NUMA 节点 (r, c) 持有的权重块为 $W_1^{(r,c)} \in \mathbb{R}^{h/R \times 4h/C}$ 。输入激活值 X 同样按照行方向切分，每个 NUMA 组内的所有节点持有相同的 $X^{(r)} \in \mathbb{R}^{b \times s \times h/R}$ (其中 b 为 batch size, s 为序列长度)。

计算流程分为两个阶段。第一阶段在每个 QPI 域 (行) 内独立完成：行 r 内的所有 C 个节点并行计算 $Y^{(r,c)} = X^{(r)} \cdot W_1^{(r,c)}$ ，得到部分输出。由于每个节点只计算了 $4h/C$ 的输出通道，这些结果在行内需要进行一次 All-Gather 操作来收集完整的 $Y^{(r)} \in \mathbb{R}^{b \times s \times 4h}$ 。这个 All-Gather 操作完全在 QPI 域内进行，利用高速互连完成。第二阶段涉及 CXL 域的通信：每个 NUMA 节点计算 $\text{GeLU}(Y^{(r)})$ 后，第二个线性层 $W_2 \in \mathbb{R}^{4h \times h}$ 采用类似的二维切分，但切分方向互换—— W_2 沿着输入维度 $4h$ 在列方向 (CXL 域) 切分，沿着输出维度 h 在行方向 (QPI 域) 切分。每个节点计算局部输出 $Z^{(r,c)} = \text{GeLU}(Y^{(r)})^{(c)} \cdot W_2^{(r,c)}$ ，其中 $\text{GeLU}(Y^{(r)})^{(c)}$ 是该节点在 CXL 域负责的 $\text{GeLU}(Y^{(r)})$ 子集。最后，在每个 QPI 域内执行 All-Reduce 操作聚合所有 C 个节点的部分结果，得到该行的最终输出 $Z^{(r)}$ 。整个流程中，主要的 All-Reduce 操作被限制在 QPI 域内，而 CXL 域主要承担激活值的子集分发和局部累加，通信量相对较小。

3.3 两级 Attention 模块并行方案

自注意力模块的两级并行方案与 MLP 模块遵循相似的原则，但充分利用了多头注意力中不同注意力头之间的计算独立性。在标准的多头注意力中，输入 X 通过 Query、Key、Value 投影矩阵 $W_Q, W_K, W_V \in \mathbb{R}^{h \times h}$ 转换为 Q、K、V 张量，然后在 h 个注意力头上并行计算注意力分数。两级并行策略将注意力头均匀地分配到二维网格上：总共 h 个注意力头被划分为 $R \times C$ 组，每个 NUMA 节点 (r, c) 负责计算 $h/(R \cdot C)$ 个注意力头。QKV 投影矩阵沿着输出维度 (注意力头维度) 在二维网格上切分，每个节点持有对应注意力头的投影权重 $W_Q^{(r,c)}, W_K^{(r,c)}, W_V^{(r,c)} \in \mathbb{R}^{h \times h/(R \cdot C)}$ 。

前向传播流程如下：首先，输入激活值 X 通过行方向切分 (QPI 域) 分发到各个 NUMA 组。在每个 NUMA 组内，所有节点持有相同的 $X^{(r)}$ 。每个节点独立计算其负责的注意力头的 Q、K、V： $Q^{(r,c)} = X^{(r)} \cdot W_Q^{(r,c)}$ ， $K^{(r,c)} = X^{(r)} \cdot W_K^{(r,c)}$ ， $V^{(r,c)} = X^{(r)} \cdot W_V^{(r,c)}$ 。由于不同节点计算的是不同的注意力头，这一步不需要跨节点通信。接下来，注意力分数计算 $A^{(r,c)} = \text{softmax}(Q^{(r,c)} \cdot K^{(r,c)T} / \sqrt{d_h}) \cdot V^{(r,c)}$ 也在每个节点上独立完成。完成注意力计算后，输出投影矩阵 $W_O \in \mathbb{R}^{h \times h}$ 采用与 MLP 中 W_2 类似的切分方式：在行方向 (QPI 域) 沿输入维度切分，在列方向 (CXL 域) 沿输出

维度切分。每个节点计算局部输出 $O^{(r,c)} = A^{(r,c)} \cdot W_O^{(r,c)}$ 。最后，在每个 QPI 域内执行 All-Reduce 操作聚合所有列节点的部分输出，同时每个 CXL 域内执行 All-Reduce 聚合所有行节点的部分输出，得到最终结果。

这种设计的关键优势在于：**注意力计算的核心部分（QKV 投影和注意力分数计算）完全不需要跨节点通信**，因为每个注意力头的计算是独立的。主要的通信集中在最后的输出投影 All-Reduce，但这个操作可以被分解为 QPI 域内的快速 All-Reduce 和 CXL 域上的低频聚合。此外，由于注意力头的计算在每个节点上是完全独立的，系统可以灵活地调整 R 和 C 的比例来适应不同的模型配置和硬件拓扑。例如，对于具有大量注意力头的模型（如 96 个头的 GPT-3），可以将更多的并行度分配给列方向（CXL 域），以充分利用跨 NUMA 的内存容量；而对于隐藏维度较大的模型，则可以增加行方向（QPI 域）的并行度，以保持高频通信在快速互连域内。

3.4 分层通信集体操作设计

两级张量并行的通信效率直接取决于集体操作（collective operations）的实现。本方案采用**分层 All-Reduce 和 All-Gather 算法**，充分利用 QPI 和 CXL 的层次化拓扑特性。对于 QPI 域内的通信（行方向），采用**环算法（Ring All-Reduce）**或**递归减半加倍算法（Recursive Halving/Doubling）**。环算法通过将数据划分为 N 个块，在环状拓扑上进行 $N-1$ 轮发送和接收，最后通过一轮 All-Gather 收集结果，其通信复杂度为 $2(N-1) \cdot \alpha + 2(N-1)/N \cdot \beta \cdot M$ （其中 α 是启动延迟， β 是每字节传输时间， M 是消息大小）。递归减半加倍算法则通过对数轮次的成对通信实现 All-Reduce，复杂度为 $\log N \cdot \alpha + 2(N-1)/N \cdot \beta \cdot M$ ，在延迟敏感的小消息场景下表现更好。对于 QPI 域内的少量设备（如 2-4 个 NUMA 节点），环算法通常能充分利用链路带宽；而对于更大规模的 QPI 域，递归算法可能更优。

对于 CXL 域内的通信（列方向），由于延迟较高但带宽尚可，采用**两阶段分层 All-Reduce**：第一阶段在每个 QPI 域内选出“主节点”，各主节点先在本地完成行方向的 All-Reduce；第二阶段，这些主节点通过 CXL 进行列方向的 All-Reduce（数据量较小，因为已经过行方向聚合）；第三阶段，主节点将结果广播回各自的 QPI 域。这种分层方式将跨 CXL 的通信量减少了 R 倍（ R 为 QPI 域内的节点数），有效摊销了 CXL 的高延迟。对于 All-Gather 操作，QPI 域内采用环算法或线性管道算法，而 CXL 域上则采用级联广播（cascade broadcast）方式，即一个节点完成接收后立即开始向其他节点转发，形成流水线效应。此外，**通信与计算重叠**是降低通信开销的关键技术：在 MLP 模块中，第一个线性层的计算可以与前一个层的 CXL 通信重叠；在 Attention 模块中，QKV 投影计算可以与上一层的输出 All-Reduce 重叠。通过精心设计调度顺序，系统可以将大部分 CXL 通信隐藏在计算背后，从而在实际执行中几乎不引入额外的延迟开销。

3.5 负载均衡与容错设计

两级张量并行方案需要精心的负载均衡策略来确保所有 NUMA 节点的计算和通信负载均衡分布。首先，在权重切分阶段，需要确保每个 NUMA 节点分到的参数量和和计算量尽可能相等。对于 MLP 模块，二维切分天然地实现了负载均衡，因为每个节点持有 $(h/R) \times (4h/C)$ 的权重，计算量与权重大小成正比。对于 Attention 模块，注意力头的分配需要处理不可整除的情况：当总注意力头数 h 不能被 $R \times C$ 整除时，剩余的应均匀分配给前几个节点，避免某个节点承担过多的计算。其次，在数据分布方面，输入序列的 batch 应均匀分配到所有行组，序列长度 s 的切分（如果使用序列并行）也应考虑负载均衡。对于变长序列场景，需要采用动态负载均衡策略，根据实际的序列长度重新分配计算任务。

容错设计对于大规模 CPU 集群尤为重要。两级张量并行可以采用**检查点（checkpointing）**和**热备（hot standby）**相结合的策略。由于每个 NUMA 节点只持有部分权重，单个节点的故障会导致整个并行组无法继续。系统可以采用周期性全局检查点的方式，每隔一定迭代保存所有节点的权重和优化器状态。当某个节点故障时，系统可以从最近的检查点恢复，并将故障节点的任务重新调度到备用节点或 CXL 内存池中的计算资源上。CXL 的内存池化特性为容错提供了额外的灵活性：关键层的权重可以在 CXL 内存池中保留冗余副本，当某个 NUMA 节点故障时，其他节点可以直接从 CXL 池读取所需的权重块，而无需等待检查点恢复。此外，系统可以实现**渐进式降级**：当检测到某个 NUMA 节点性能下降（如热节流）时，可以动态减少该节点的任务分配，将部分计算迁移到通过 CXL 连接的其他节点，虽然性能会有所下降，但整个系统可以继续运行而不会中断。

4. 通信成本模型与性能分析

4.1 1D 张量并行通信成本分析

在标准的 1D 张量并行中，每个 Transformer 层需要进行两次 All-Reduce 操作（MLP 和 Attention 各一次）。对于隐藏维度为 h 、batch size 为 b 、序列长度为 s 的模型，每次 All-Reduce 传输的数据量为 $b \cdot s \cdot h$ 个元素（假设使用 bfloat16 格式，每个元素 2 字节）。All-Reduce 的通信时间可以使用标准的集体通信成本模型来估计。采用环算法（Ring All-Reduce），在 P 个并行设备上，All-Reduce 的时间为：

$$T_{1D-AR} = 2(P-1) \cdot \alpha + \frac{2(P-1)}{P} \cdot \frac{M}{B}$$

其中 α 是网络启动延迟 (latency)， $M = b \cdot s \cdot h \cdot 2$ 是消息大小 (字节)， B 是设备间可用带宽。对于跨 NUMA 的 1D 张量并行， B 对应于 QPI/UPI 的有效带宽 (约 10-25 GB/s)， α 约为 200-400 纳秒。一个包含 L 层的 Transformer 模型在每个训练步骤中的总通信时间为：

$$T_{1D-total} = 4L \cdot T_{1D-AR}$$

(系数 4 是因为前向和反向传播各需要两次 All-Reduce)。当 P 增大时，带宽项 $\frac{2(P-1)}{P} \cdot \frac{M}{B}$ 趋近于 $2M/B$ ，但延迟项 $2(P-1)\alpha$ 线性增长，这使得 1D 张量并行在设备数较多时迅速被延迟主导。更重要的是，当并行组跨越 QPI 和 CXL 两个层级时，所有 All-Reduce 操作都必须 traverses 最慢的 CXL 链路，导致带宽 B 被拉低到 CXL 的水平 (约 10-25 GB/s)，而延迟 α 增加到 500 纳秒以上，这使得 1D 张量并行在多 NUMA+CXL 架构上效率极低。

4.2 两级张量并行通信成本模型

两级张量并行通过将 All-Reduce 操作分解为 QPI 域内和 CXL 域上两个层次，显著降低了整体通信成本。对于 MLP 模块，前向传播涉及以下通信操作：(1) 第一个线性层后，在每个 QPI 域 (R 个节点) 内进行 All-Gather，收集 C 个列节点的输出；(2) 第二个线性层后，在每个 QPI 域内进行 All-Reduce，聚合 C 个列节点的部分结果；(3) Attention 模块的输出 All-Reduce 同样限制在 QPI 域内。假设使用环算法，QPI 域内的 All-Reduce 时间为：

$$T_{QPI-AR} = 2(C-1) \cdot \alpha_{QPI} + \frac{2(C-1)}{C} \cdot \frac{M/R}{B_{QPI}}$$

其中 $\alpha_{QPI} \approx 150 - 200$ 纳秒， $B_{QPI} \approx 20 - 50$ GB/s， M/R 是因为输入数据在行方向已被切分。对于 CXL 域上的通信，主要发生在每 R 个迭代一次的跨域梯度同步（如果使用异步优化器）或激活值交换：

$$T_{CXL-sync} = 2(R-1) \cdot \alpha_{CXL} + \frac{2(R-1)}{R} \cdot \frac{M_{grad}}{B_{CXL}}$$

其中 $\alpha_{CXL} \approx 350 - 600$ 纳秒， $B_{CXL} \approx 10 - 30$ GB/s， M_{grad} 是跨域同步的梯度数据量。由于 CXL 同步的频率远低于每层的 All-Reduce（可以每 k 个迭代进行一次， $k \gg 1$ ），其摊销成本远低于 QPI 通信。在每迭代基础上，两级张量并行的总通信时间为：

$$T_{2D-total} \approx 4L \cdot T_{QPI-AR} + \frac{4L}{k} \cdot T_{CXL-sync}$$

当 k 足够大时（如使用大批量梯度累积），第二项趋近于零，这意味着系统的主要通信被限制在 QPI 域内。相比 1D 张量并行中所有通信都必须 traverses CXL 链路的情况，两级方案可以将有效通信带宽提升 2 - 5 倍，延迟降低 2 - 3 倍。

4.3 与 2.5D/3D 张量并行的比较

两级张量并行方案与 2.5D 和 3D 张量并行在降低通信复杂度方面有相似的动机，但采用了不同的实现路径。2.5D 张量并行通过引入深度维度 D 来减少通信量，其通信复杂度为 $O(\sqrt{P/D})$ ，但需要额外的内存来存储 D 份数据副本。3D 张量并行实现了最优的 $O(P^{1/3})$ 通信复杂度，但要求设备数为完全立方数且拓扑为三维网格。两级张量并行本质上是一种**拓扑感知的 2D 张量并行变体**，其通信复杂度与标准 2D SUMMA 算法相同，为 $O(\sqrt{P})$ ，但通过将通信映射到不同速度的互连层级上，实现了”延迟隐藏”的效果。

方案	通信复杂度	内存冗余	拓扑要求	异构网络适配
1D TP	$O(P)$	高	任意	差
2D TP (SUMMA)	$O(\sqrt{P})$	低	二维网格	中等
2.5D TP	$O(\sqrt{P/D})$	D 倍	$P = Q^2 D$	中等
3D TP	$O(P^{1/3})$	最低	三维网格	差
两级 TP (本方案)	$O(\sqrt{P})$ (QPI 域内)	低	层次化 NUMA+CXL	优秀

两级方案的独特优势在于它**不需要额外的内存冗余**（不像 2.5D 需要复制数据），且对设备数量没有严格的数学限制（不像 3D 要求完全立方数）。它通过通信调度而非算法复杂度来优化性能，这使得它在实际的异构 CPU 集群上更具实用性。此外，两级方案可以与 2.5D 的思想结合：在 QPI 域内使用深度维度 D 进一步减少通信，同时在 CXL 域上保持低频同步，从而在内存允许的情况下获得更低的通信复杂度。

4.4 性能瓶颈与优化策略

尽管两级张量并行显著改善了通信效率，系统仍可能面临多种性能瓶颈。首先，**QPI 域内的带宽饱和**是一个潜在问题：当 C （列数，即每个 QPI 域内的节点数）较大时，QPI 域内的 All-Reduce 可能成为瓶颈，因为所有节点共享有限的 QPI 带宽。解决这个问题的方法包括增加 QPI 域的数量（增加 R ）或减少每个 QPI 域的节点数，以及使用更高版本的 UPI（如 Intel 的 UPI 2.0 提供更高带宽）。其次，**CXL 延迟敏感操作**需要特别关注：虽然大部分 CXL 通信可以与计算重叠，但某些同步操作（如全局梯度规约）可能无法完全隐藏延迟。优化策略包括使用异步优化器（如 Async SGD 或 Local SGD），允许各 QPI 域独立进行多个步骤的本地更新后再进行全局同步，从而摊销 CXL 延迟。第三，**负载不均衡**可能由多种因素引起：不同 Transformer 层的计算量不同（如 Attention 的 $O(s^2)$ 复杂度），变长序列导致不同 batch 的处理时间不同，以及系统层面的进程调度干扰。解决方案包括动态负载均衡、计算与通信的流水线重叠，以及使用 NUMA-aware 的进程绑定策略将计算线程固定到本地核心。最后，**内存带宽瓶颈**在 CPU 平台上尤为突出：虽然张量并行减少了每个节点的数量，但激活值和 KV-cache 的内存访问仍可能饱和本地 DRAM 带宽。优化策略包括使用 CXL 内存作为激活值的溢出存储、采用激活值重计算（activation checkpointing）来减少内存占用，以及使用低精度数据类型（如 INT8 或 FP8）来减少内存带宽需求。

5. 相关研究与前沿进展

5.1 异构网络下的自适应张量并行

自适应张量并行 (Adaptive Tensor Parallelism, ATP) 是近年来针对异构网络环境提出的重要研究方向。ATP 的核心思想是通过自动搜索最优的设备网格 (device mesh) 和张量并行策略来适应特定的硬件拓扑。Cheng 等人提出的 ATP 框架引入了**层次化通信矩阵 (Hierarchical Communication Matrix)** 的概念，将现代加速器互连的多层结构（如节点内 NVLink 和节点间 InfiniBand）形式化为数学模型。该矩阵包含多个层次，每个层次包含一个 P2P 通信矩阵和组带宽 (group bandwidth)，能够精确描述不同设备组之间的通信能力。基于这个模型，ATP 可以估计任意设备网络上 All-Reduce 操作的带宽，并自动选择通信时间最短的配置。

ATP 的搜索空间包括所有可能的二维设备网格切分方式（行优先或列优先），以及不同的张量切分维度。对于每个候选配置，ATP 使用层次化通信矩阵计算 All-Reduce 的有效带宽，然后选择总通信时间最小的方案。实验表明，在具有复杂 PCIe 和 NVLink 拓扑的多 GPU 服务器上，ATP 相比固定的 1D 张量并行可以获得显著的性能提升。对于 CPU

NUMA+CXL 架构, ATP 的框架可以直接扩展: 将 QPI/UPI 域建模为层次化矩阵的低层 (高带宽、低延迟), 将 CXL 域建模为高层 (较低带宽、较高延迟), 然后搜索最优的二维切分方案。这种自动化的方法可以替代手工设计的并行策略, 特别适用于拓扑复杂或动态变化的 CPU 集群。然而, ATP 的搜索过程本身需要时间, 对于静态部署可以离线完成, 但对于动态负载场景则需要更高效的在线搜索算法。

5.2 分层模型并行与 NUMA 感知推理

分层模型并行 (Hierarchical Model Parallelism) 的概念在 CPU 平台上已有实践, 特别是在 ARM 多核处理器的 3D-CNN 推理优化中。针对 ARM 服务器上的多 NUMA 架构, 研究者提出了两级分区策略: 首先在通道维度上对 3D-CNN 模型进行粗粒度切分, 将不同的通道组分配到不同的 NUMA 节点 (inter-node partition); 然后在每个 NUMA 节点内部, 进一步将输入张量细分为缓存友好的小块 (cache-friendly partition), 以消除线程竞争。这种策略的核心优势在于**计算与数据的同地化 (co-location)**: 通过线程绑定 (thread pinning) 确保计算线程在其数据所在的 NUMA 节点上运行, 从而消除大部分远程内存访问。实验表明, 在 128 线程的 ARM 服务器上, 这种分层方法相比朴素实现可以获得 11 倍到 50 倍的性能提升。

对于 Transformer 模型的 CPU 推理, 类似的 NUMA 感知策略已被集成到 vLLM 和 DeepSpeed 等框架中。这些框架在启动时自动检测系统的 NUMA 拓扑 (NUMA 节点数、每个节点的核心数、节点间的距离矩阵), 然后选择张量并行或流水线并行。例如, 当检测到对称的 NUMA 配置 (每个插槽 2 或 4 个 NUMA 节点) 时, 系统选择张量并行; 而对于非对称配置 (每个插槽 3 或 6 个 NUMA 节点), 则选择流水线并行以获得更好的负载均衡。这种自动化的拓扑感知策略可以最小化跨 NUMA 流量, 最大化内存本地性和缓存利用率。对于 CXL 内存池化的支持, 最新的研究开始探索如何将 CXL 内存作为额外的内存层级来管理大模型的 KV-cache 和激活值, 通过 NUMA 策略工具 (如 numactl) 将不频繁访问的数据迁移到 CXL 内存, 从而释放本地 DRAM 给计算密集型任务。

5.3 CXL 内存池化与大模型训练

CXL 内存池化技术为大模型训练提供了新的可能性, 特别是在 CPU 平台上。传统的 GPU 训练受限于单卡显存容量, 而 CXL 允许通过内存池动态分配和共享内存资源, 从而支持更大的模型和更长的序列。研究表明, CXL 内存访问延迟约为本地 DRAM 的 2.2 倍 (约 200-400 纳秒), 但提供了内存容量的大幅扩展和更灵活的资源分配。对于张量并行而言, CXL 内存池可以用于存储模型参数和优化器状态, 而计算则在本地 CPU 上进行, 实现了计算和内存的解耦。这种方式类似于 CPU-offloading 技术, 但 CXL 提供了比传统 RDMA 更低的延迟和硬件级的一致性支持。

在实际部署中, CXL 内存池化的性能取决于多个因素: CXL 控制器的实现质量 (ASIC vs FPGA)、PCIe 版本 (Gen5 vs Gen6)、是否经过交换机、以及工作负载的访问模式。研究发现, CXL 内存的带宽 saturate 于重负载的多线程场景, 但对于稀疏或不规则的访问模式, 延迟的不稳定性可能成为问题。特别是对于 LLM 训练中的 All-Reduce 操作, 如果数据需要通过 CXL 传输, 较高的延迟可能显著影响性能。因此, 最新的研究提出了**分层内存管理策略**: 将热数据 (如当前层的权重和激活值) 保留在本地 DRAM 中, 而将冷数据 (如其他层的权重、优化器状态、KV-cache) 存储在 CXL 内存池中。这种策略可以最大化本地 DRAM 带宽的利用率, 同时利用 CXL 的容量优势来支持更大的模型。对于两级张量并行方案, CXL 内存池可以作为第二级并行 (TP-2) 的存储后端, 参数在 CXL 池中共享, 而高频的 All-Reduce 操作则在 QPI 域内的本地 DRAM 中完成。

5.4 网络感知的并行策略放置

网络感知的并行策略放置 (Network-Aware Parallelism Placement) 是分布式训练领域的重要研究方向, 其核心思想是根据底层网络拓扑的特点来优化并行维度的放置顺序。在 Llama 3 的 405B 模型训练中, Meta 采用了 [TP, CP, PP, DP] 的并行层次顺序, 将需要最高带宽和最低延迟的张量并行 (TP) 放在最内层 (同一服务器内), 而将可以容忍更高延迟的数据并行 (DP) 放在最外层 (跨多跳网络)。这种层次化的放置策略确保了每种并行模式的通信需求与其可用的网络资源相匹配。

对于 CPU 多级互连架构, 同样的原则适用但需要更精细的建模。ATP 框架提出的层次化通信矩阵为这种建模提供了工具: 通过将 QPI/UPI 域建模为高带宽层, CXL 域建模为低带宽层, 可以自动推导出最优的并行放置策略。研究表明, 在异构

网络环境下，优先使用节点内张量并行（最小化单请求延迟），跨节点使用流水线并行（减少有限带宽上的通信），避免跨区域通信（因超低带宽），可以显著优于简单的对称并行策略。对于两级张量并行方案，网络感知放置意味着将 QPI 域内的 All-Reduce 操作设计为高频率、小数据量的同步，而将 CXL 域上的通信设计为低频率、大数据量的批量传输，从而最大化两种互连层级的利用率。此外，**拓扑感知的 All-Reduce 算法**（如针对 BCube 拓扑的 BML 算法、针对 2D Torus 的 2D-Torus All-Reduce）可以进一步优化分层通信的性能，通过利用拓扑的对称性和冗余链路来减少通信延迟。

6. 实现考虑与部署建议

6.1 软件框架支持

实现两级张量并行需要对现有的分布式训练框架进行扩展。PyTorch 的分布式张量（DTensor）和 DeviceMesh 抽象为实现两级并行提供了良好的基础。DeviceMesh 允许将设备组织成多维网格，并自动处理不同维度上的通信组创建。通过定义一个二维的 DeviceMesh（行对应 QPI 域，列对应 CXL 域），可以方便地实现分层 All-Reduce 和 All-Gather 操作。PyTorch 的 `parallelize_module` 函数可以与自定义的并行策略结合，通过在 MLP 和 Attention 层上应用特定的切分规则来实现两级并行。对于更低层的通信优化，oneCCL（Intel oneAPI Collective Communications Library）提供了针对 Intel 互连架构（包括 QPI/UPI 和 CXL）优化的集体操作实现，可以替代标准的 Gloo 或 NCCL 后端。

DeepSpeed 的集成方案提供了更高级的抽象，包括 ZeRO 优化器、流水线并行和张量并行的组合。两级张量并行可以作为 DeepSpeed 的一种新的并行模式进行扩展，与现有的 3D 并行（TP+PP+DP）结合形成 4D 并行（TP1+TP2+PP+DP）。在这种方案中，TP1 限制在 QPI 域内，TP2 跨越 CXL 域，PP 用于层间并行，DP 用于数据并行。DeepSpeed 的通信调度器可以被扩展为支持分层通信，自动将 All-Reduce 操作路由到适当的互连层级。此外，Intel 的 IPEX-LLM 框架提供了针对 CPU 的优化算子库（包括 AMX 加速的 GEMM 内核），可以与两级张量并行结合使用，以最大化 CPU 的计算效率。

6.2 硬件配置建议

对于两级张量并行的硬件配置，建议采用以下原则：（1）**QPI/UPI 域内最大化带宽**：选择具有多个 UPI 链路的高端 CPU（如 Intel Xeon Platinum 系列），确保同一 QPI 域内的节点之间具有充足的带宽。对于双路服务器，确保两个插槽之间有至少两条 UPI 链路直连。（2）**CXL 链路带宽与延迟平衡**：选择基于 ASIC 的 CXL 控制器（而非 FPGA），以获得更低的延迟和更高的带宽。优先使用 PCIe Gen5 或 Gen6 的 CXL 设备，确保 x16 链路带宽不低于 64 GB/s。（3）**内存容量规划**：本地 DRAM 应足够容纳至少一层 Transformer 的权重和激活值（对于大模型，可能需要几十 GB），而 CXL 内存池则用于存储剩余层的权重和优化器状态。（4）**NUMA 拓扑优化**：在 BIOS 中启用 Sub-NUMA Clustering（SNC）模式，将每个插槽进一步划分为多个 NUMA 节点，以提供更细粒度的内存本地性。

典型的推荐配置包括：双路或四路 Intel Xeon 6 处理器（支持 AMX 和 CXL），每个插槽配备 256-512 GB 本地 DRAM，通过 CXL 交换机连接 4-8 TB 的 CXL 内存池。对于 All-Reduce 操作，QPI 域内的带宽目标应不低于 20 GB/s（延迟 <200 ns），CXL 域内的带宽目标应不低于 10 GB/s（延迟 <500 ns）。这种配置可以支持数十亿到数百亿参数规模的模型进行高效训练。对于推理场景，配置可以更加灵活，使用较少的本地 DRAM（因为不需要存储优化器状态），更多的 CXL 内存来容纳大模型的权重。

6.3 性能调优指南

在实际部署两级张量并行时，需要进行系统性的性能调优：（1）**并行度调优**：通过网格搜索或贝叶斯优化找到最优的 $R \times C$ 配置。通常， C （QPI 域内的节点数）应较小（2-4），以确保 All-Reduce 在快速互连上完成；而 R （CXL 域内的 QPI 组数）可以较大，以充分利用 CXL 的内存容量。（2）**通信频率调优**：调整全局同步的频率（如每 k 个迭代进行一次跨 CXL 同步），在模型收敛速度和通信开销之间找到平衡。对于大 batch 训练，可以增加 k 来摊销 CXL 延迟。（3）**内存分配策略**：使用 `numactl` 或 `libnuma` 将热数据（当前计算层的权重和激活值）绑定到本地 DRAM，冷数据（其他层的权重）分配到 CXL 内存。Linux 内核的 NUMA 平衡功能可以辅助这种自动迁移。（4）**线程绑定与调度**：使用 `taskset` 或 OpenMP 的 `OMP_PLACES` 将计算线程固定到本地 NUMA 节点，避免操作系统调度导致的远程内存访问。对于超线程（SMT）架构，需要考虑逻辑核心与物理核心的映射关系。（5）**性能监控与分析**：使用 Intel VTune、AMD uProf 或

Linux perf 工具监控 NUMA 命中率、远程访问比例、内存带宽利用率等关键指标。如果远程访问比例超过 10%，则需要重新审视数据放置策略。

6.4 未来发展方向

两级张量并行方案的未来发展方向包括：(1) **动态拓扑自适应**：随着 CXL Fabric 技术的发展，系统拓扑可能在运行时动态变化（如 CXL 内存的热插拔）。未来的两级并行方案需要支持动态重新配置，在不停机的情况下调整并行网格和通信路由。(2) **与 CXL 3.0/4.0 的内存共享集成**：CXL 3.0 引入的硬件级内存共享允许多个主机同时访问同一段物理内存。这可以进一步优化两级并行：多个 QPI 域可以共享同一份 CXL 内存中的权重副本，减少内存冗余和同步开销。(3) **扩展到 GPU+CXL 混合架构**：随着 GPU 也开始支持 CXL（如 NVIDIA 的 Grace Hopper 架构），两级张量并行可以扩展到 CPU+GPU 混合环境，其中 GPU 负责计算密集型任务，CPU 管理 CXL 内存池和低频通信。(4) **自动化并行策略搜索**：借鉴 ATP 和 Alpa 等框架的思想，开发针对 CPU NUMA+CXL 拓扑的自动并行策略搜索工具，通过分析硬件拓扑和模型特性自动推导出最优的两级切分方案。(5) **与量化、稀疏化技术结合**：将两级张量并行与低精度量化（如 INT8/FP8）、结构化稀疏化（如 N:M 稀疏）和条件计算（如 MoE）结合，进一步减少通信量和内存占用，支持更大规模的模型训练。

参考文献

- [1] Shoeybi M, Patwary M, Puri R, et al. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053, 2019.
- [2] Xu Q, Zhang H, Gao G, et al. An Efficient 2D Method for Training Super-Large Deep Learning Models. IPDPS, 2023.
- [3] Wang B, Li Q, Zhang Y, et al. Tesseract: Parallelize the Tensor Parallelism Efficiently. ICPP, 2022.
- [4] Bian Z, Liu H, Wang B, et al. Maximizing Parallelism in Distributed Training for Huge Neural Networks. arXiv:2105.14450, 2021.
- [5] Li S, Xue F, Baranwal C, et al. Sequence Parallelism: Long Sequence Training from System Perspective. ACL, 2023.
- [6] Korthikanti V, Casper J, Lym S, et al. Reducing Activation Recomputation in Large Transformer Models. arXiv:2205.05198, 2022.
- [7] Cheng W, Wu R, Zhang S, et al. ATP: Adaptive Tensor Parallelism for Foundation Models. arXiv:2301.08658, 2023.
- [8] Solomonik E, Demmel J. Communication-Optimal Parallel 2.5D Matrix Multiplication and LU Factorization Algorithms. Euro-Par, 2011.
- [9] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. NeurIPS, 2017.
- [10] Das Sharma D, et al. An Introduction to the Compute Express Link (CXL) Interconnect. ACM Computing Surveys, 2024.